Working Thesis Skeleton for TAMER-OP

Encoding-Centered Multiparameter Persistence via Finite Poset Models

Erik Mendes Novak

March 10, 2026

Contents

1 Introduction **3**
1.1 Possible opening paragraph . . . 3
1.2 Proposed contribution list . . . 4

2 Background **4**
2.1 Posets, functors, and persistence modules . . . 4
2.2 Tame modules, finite encodings, and fringe presentations . . . 4
2.3 Why multiparameter persistence is hard . . . 5
2.4 Minimal software landscape . . . 5
2.5 Conventions and notational setup . . . 5

3 Overview of TAMER-OP as an encoding-centered system **5**
3.1 The main pipeline . . . 6
3.2 Architectural layering . . . 6
3.3 Why the finite encoding step is the organizing principle . . . 6

4 Pre-encoding objects and data interfaces **6**
4.1 Finite-poset presentations: finite fringes and indicator data . . . 6
4.2 $\mathbb{Z}^n$ front-end: flanges and finitely determined data . . . 7
4.3 $\mathbb{R}^n$ front-end: PL fringes and box presentations . . . 7
4.4 Raw data, filtrations, and graded complexes . . . 7
4.5 File-oriented ingestion and reproducible front ends . . . 7
4.6 What this chapter should accomplish . . . 7

5 The encoding step **8**
5.1 Statement of the encoding problem . . . 8
5.2 Region signatures and the finite-poset idea . . . 8
5.3 Encoding finite fringe data . . . 8
5.4 Encoding $\mathbb{Z}^n$ flanges . . . 8
5.5 Encoding $\mathbb{R}^n$ PL and box presentations . . . 9
5.6 Data-to-encoding via the ingestion layer . . . 9
5.7 Common encodings for multiple modules . . . 9
5.8 Compiled encodings, point location, and evaluation . . . 9
5.9 Geometry attached to an encoding map . . . 9
5.10 Correctness, diagnostics, and failure modes . . . 10
5.11 Complexity and why this chapter is the thesis core . . . 10

6 Post-encoding algebra on finite posets **10**
6.1 Encoded modules and morphisms as the internal working language 11
6.2 Abelian-category constructions in code 11
6.3 Indicator resolutions, projective covers, and injective hulls 11
6.4 Derived functors and complex-level structure 11
6.5 Change of posets, transport, and comparison 11
6.6 What computational opportunities this opens 11

7 Invariants and summaries after encoding **12**
7.1 Restricted Hilbert and rank-type invariants 12
7.2 Slice-based invariants and distances 12
7.3 Geometry of encoding regions and weighted summaries 12
7.4 Mobius inversions, signed measures, and Euler-type summaries 13
7.5 Multiparameter persistence images and higher summaries 13
7.6 How much theory to include here 13

8 Serialization, featurization, and visualization **13**
8.1 Serialization and reproducibility 13
8.2 Featurization as an experiment layer 13
8.3 From geometric summaries to features and visualization 14
8.4 Why these layers matter scientifically 14
8.5 Visualization engine 14

9 Performance engineering and benchmarks **14**
9.1 Benchmark philosophy and fairness 15
9.2 Where time is spent internally 15
9.3 Microbenchmarks 15
9.4 End-to-end workflow benchmarks 15
9.5 Comparison with MULTIPERS 15
9.6 Discussion of tractability 16

10 Conclusion and future work **16**

A Section-to-codebase map **16**

B Planned figures and tables **17**

C Benchmark checklist **17**

D Bibliography notes **17**

Working brief

This skeleton is organized around the claim that *encoding is the central computational move* in TAMER-OP: one begins with pre-encoding objects (finite fringes, flanges over $\mathbb{Z}^n$, PL fringes or box presentations over $\mathbb{R}^n$, and raw data), performs a finite encoding step, and then works entirely in the finite-poset world where homological algebra, categorical constructions, invariants, serialization, featurization, and visualization become uniform and programmable. The mathematical framework should be tied explicitly to Miller's theory of modules over posets and finite encodings [Mil20], while the implementation chapters should track the actual architecture of the current code base and its workflow/API layers.

- **Target length.** Roughly 30–40 pages total. A plausible distribution is: Introduction (1–2), Background (3–5), system overview (2), pre-encoding (4), encoding proper (8–10), post-encoding algebra (5–6), invariants (3–4), serialization/featurization/visualization (4–5), benchmarks (5–6), conclusion (1–2).
- **Purpose.** Make the document implementation-facing: every section should be something that can be drafted independently from the code and stitched together later.
- **Keep brief.** Do *not* spend many pages re-teaching standard homological algebra, category theory, or classical persistent homology. Spend those pages instead on the library's distinctive contribution: the finite encoding layer and everything it unlocks.
- **Deliverable.** A thesis that reads as both a mathematical narrative and a systems paper: it should explain *what* is represented, *how* it is encoded, *what computations become possible afterward*, and *why this architecture matters in practice*.

1 Introduction

- **Target length.** 1–2 pages.
- **Purpose.** State the problem, the main thesis, and the core contributions without front-loading too much theory. This should be substantially shorter than the current draft's long opening arc.
- **Include.** A short motivation from multiparameter persistence: 1-parameter persistence enjoys a strong structure theory, but in multiple parameters one must choose what algebraic structure to preserve and what computational compromises to make [CZ09, Les15, CDSGO16].
- **Include.** A crisp statement of the library's design thesis: TAMER-OP is organized around finite encoding, meaning that infinite or geometric parameter spaces are first compressed into finite posets, after which the library performs exact algebraic and homological computations there.
- **Include.** A one-paragraph contributions list. Suggested bullets: (i) unified front-end formats and data ingestion; (ii) explicit finite encoding constructors over $\mathbb{Z}^n$ and $\mathbb{R}^n$; (iii) post-encoding module/category/homological algebra; (iv) invariant, serialization, featurization, and visualization layers; (v) benchmark evidence for tractability and competitiveness.
- **Include.** A short roadmap paragraph telling the reader what each major section does.
- **Keep brief.** Do not prove anything here. Do not give a long survey of the literature. Do not yet explain all implemented invariants or all derived functors.
- **Possible figures/tables.** One architecture figure can already appear here if desired: *data/presentation* $\to$ *encoding* $\to$ *finite-poset computations*, with serialization, featurization, and visualization drawn as cross-cutting layers.

1.1 Possible opening paragraph

- **Include.** Start from the tension between generality and computability in multiparameter persistence. Emphasize that many libraries stop at barcodes, ranks, or machine-learning summaries, whereas TAMER-OP aims to preserve enough algebraic structure to support exact categorical and homological computations after a finite encoding step.

- **Comparison point.** Contrast carefully with tools such as RIVET and MULTIPERS: those systems are important comparison points, but the thesis should frame the distinction as one of *architecture and computational scope*, not as a dismissive software ranking [LW15, RIVET, LS24].

1.2 Proposed contribution list

- **Include. Mathematical contribution of the implementation:** explicit finite encoding serves as a uniform gateway between tame modules over large parameter spaces and finite-poset module computations in the sense of Miller's framework [Mil20].
- **Include. Systems contribution:** the library integrates pre-encoding representations, encoding algorithms, categorical/homological post-processing, invariant computation, serialization, and experiment-ready featurization into one workflow surface.
- **Include. Practical contribution:** the same encoded object can be used for algebraic analysis, statistical summaries, feature extraction, and exploratory visualization without rebuilding separate computational pipelines.

2 Background

- **Target length.** 3–5 pages total. This section should be drastically shorter than the current write-up's long background chapter.
- **Purpose.** Provide only the mathematical language needed for the rest of the thesis. The reader should leave this section knowing what a poset module is, what "tame" and "finitely encoded" mean at a high level, and why this matters computationally.
- **Keep brief.** Treat this as compressed prerequisite material. Whenever a concept is standard and not central to the implementation, define it, cite it, and move on.

2.1 Posets, functors, and persistence modules

- **Purpose.** Introduce modules over posets in the categorical language used throughout the library.
- **Include.** Posets as thin categories; persistence modules as functors $Q \to \mathbf{Vect}_{\Bbbk}$; morphisms as natural transformations; objectwise kernels/cokernels in the functor category.
- **Include.** A short explanation of why this categorical viewpoint matters computationally: once the index category is finite, many constructions reduce to matrix algebra over fibers and structure maps.
- **Keep brief.** No long proofs. Cite standard references such as Mac Lane and Weibel where appropriate [Mac98, Wei94].
- **Possible figures/tables.** A commuting-square diagram for a module morphism; a small finite poset with attached vector spaces and maps.

2.2 Tame modules, finite encodings, and fringe presentations

- **Purpose.** Compress the Miller background to the exact ideas the thesis uses later.
- **Include.** A high-level description of constant subdivisions, finite encodings, fringe presentations, and indicator resolutions, framed as equivalent "tameness" viewpoints in the underlying theory [Mil20].

- **Include.** Explain the computational moral: finite encoding is not merely a theoretical convenience but the step that turns large geometric modules into finite combinatorial objects amenable to exact linear algebra and homological algebra.
- **Keep brief.** Do not restate the full Miller development. One paragraph on each of finite encoding, fringe/flange presentation, and syzygy-theorem-style equivalence is enough.
- **Possible figures/tables.** A schematic implication diagram: finite encoding $\Longleftrightarrow$ fringe presentation $\Longleftrightarrow$ indicator resolutions, with a note that the implementation exploits these equivalences.

2.3 Why multiparameter persistence is hard

- **Purpose.** Motivate the architectural choice to encode, rather than attempt to work directly with geometric parameter spaces at every stage.
- **Include.** Very brief discussion of the failure of a barcode-style classification in the multiparameter setting, the explosion of comparable parameter pairs, and the tension between exact structure and practical computation [CZ09, Les15, CDSGO16].
- **Include.** Mention that many useful invariants and summaries exist, but they often interact weakly with category-theoretic and homological constructions unless one preserves a richer internal algebraic model.
- **Keep brief.** This is motivation, not a survey chapter.

2.4 Minimal software landscape

- **Purpose.** Prepare the later benchmark and comparison sections.
- **Include.** A short paragraph each on RIVET and MULTIPERSas comparison points. RIVET should appear as a visualization- and rank-oriented reference for low-dimensional exploration, while MULTIPERSshould appear as a modern multiparameter pipeline for invariants and machine-learning workflows [LW15, RIVET, LS24].
- **Include.** State that the thesis is not claiming to replace these tools in every regime; rather, it is presenting a different architecture whose strength is the explicit finite encoding step and the algebraic computations that follow from it.

2.5 Conventions and notational setup

- **Purpose.** Pin down notation once so later implementation chapters read cleanly.
- **Include.** Notation for Q and P as posets, M, N for modules, π for encoding maps/classifiers, $\Bbbk$ for the coefficient field, and the distinction between the ambient parameter domain and the finite encoding poset.
- **Include.** A sentence that exact arithmetic over $\mathbb{Q}$ will play an important implementation role, even when real or prime-field backends are also supported.
- **Deliverable.** End the section with a transition sentence: from here onward, the thesis focuses on *how* TAMER-OP turns these abstract objects into a unified computational system.

3 Overview of TAMER-OP as an encoding-centered system

- **Target length.** About 2 pages.

- **Purpose.** Give the reader a map of the library before diving into the details. This chapter should bridge the mathematics of the background section to the concrete code architecture.

3.1 The main pipeline

- **Include.** Organize the entire thesis around the three-stage story: **pre-encoding** (finite fringes, flanges, PL fringes, data), **encoding** (construction of a finite poset and classifier), and **post-encoding** (module operations, derived functors, invariants, transport, comparisons).
- **Include.** Explain that serialization, featurization, and visualization are not auxiliary afterthoughts but parallel layers that interact with every stage of the pipeline.
- **Possible figures/tables.** A full-page diagram mapping user-facing workflow calls to internal layers: `DataFileIO/DataIngestion` → `Workflow.encode` → `Results` → `Invariants/DerivedFunctors/`

3.2 Architectural layering

- **Include.** Briefly name the main module families: core runtime and caches; finite-poset and representation layers; encoding layers over $\mathbb{Z}^n$ and $\mathbb{R}^n$; module/category/homological algebra layers; invariants; ingestion; workflow; serialization; featurization; visualization.
- **Include.** Make explicit that the thesis is not a file-by-file code tour. Instead, module names should be used only to anchor the mathematical and algorithmic narrative.
- **Deliverable.** A short paragraph that says later chapters will follow the same order as the pipeline itself.

3.3 Why the finite encoding step is the organizing principle

- **Include.** State the central argument of the thesis in one place: once an object is encoded into a finite poset, the entire rest of the library — exact linear algebra, categorical constructions, resolutions, derived functors, invariants, featurizers, transport, and visualization — can be implemented against a single finite working model.
- **Include.** This is the conceptual through-line that should tie the whole thesis together. Return to it explicitly at the start and end of later sections.

4 Pre-encoding objects and data interfaces

- **Target length.** About 4 pages.
- **Purpose.** Describe the front-end mathematical objects the library accepts before any finite encoding is built. This section should make clear that the system is intentionally multi-front-end but single-back-end.

4.1 Finite-poset presentations: finite fringes and indicator data

- **Include.** Introduce finite posets, upsets, downsets, and finite fringe modules as the most direct interface when a finite combinatorial model is already known.
- **Include.** Explain why this layer matters even in a thesis centered on encoding: it provides the basic correctness model and the simplest test bed for categorical and homological algorithms.

- **Evidence / examples.** One small illustrative finite-poset example showing an upset/downset presentation and the resulting module.

4.2 $\mathbb{Z}^n$ front-end: flanges and finitely determined data

- **Include.** Describe flange presentations as the $\mathbb{Z}^n$-specific analogue of fringe presentations in Miller's theory, and explain why they are a natural interface for finitely determined multiparameter modules [Mil20].
- **Include.** Emphasize that these objects already carry enough structure to support a direct encoding into a finite signature/region poset, making them the cleanest discrete front-end.
- **Possible figures/tables.** A picture of a few flats/injectives in $\mathbb{Z}^2$ and their overlap pattern.

4.3 $\mathbb{R}^n$ front-end: PL fringes and box presentations

- **Include.** Describe the real-parameter front-end in two layers: general PL or polyhedral objects, and the important axis-aligned box specialization.
- **Include.** Explain why the box-specialized backend is important computationally: it captures a large practical class while enabling faster region construction and point location than full general polyhedral machinery.
- **Include.** Tie this directly to Miller's geometric perspective on real-parameter modules and finite encodings [Mil20, Mil23, KS18].

4.4 Raw data, filtrations, and graded complexes

- **Include.** Introduce the data ingestion side as a major addition relative to the earlier thesis draft: point clouds, graphs, images, embedded planar graphs, graded complexes, and simplex-tree-style multicritical data are all first-class front-end objects.
- **Include.** Explain that the ingestion layer supports named filtration families, typed filtration specifications, construction budgets, and multiple output stages before encoding.
- **Include.** Make clear that this is one of the library's most important practical contributions: it closes the gap between raw topological data analysis inputs and the encoded algebraic world.
- **Possible figures/tables.** A pipeline figure from point cloud / graph / image to graded complex to encoded module.

4.5 File-oriented ingestion and reproducible front ends

- **Include.** Briefly explain the role of file adapters and typed options: front-end objects can come from JSON artifacts, delimited tables, Ripser-style formats, and other serialized sources, all normalized into typed ingestion objects before entering the workflow.
- **Include.** This subsection can be short, but it should set up the later serialization and experiment sections by emphasizing that reproducible front ends are part of the system design from the start.

4.6 What this chapter should accomplish

- **Deliverable.** By the end of the chapter, the reader should understand that TAMER-OP accepts many mathematically equivalent or practically useful entry formats, but all of them are funneled toward one central step: building a finite encoding.

5 The encoding step

- **Target length.** 8–10 pages. This is the core chapter of the thesis and should be the single longest section.
- **Purpose.** Explain, in implementation detail, how the library turns pre-encoding objects into finite-poset models. This is the main differentiator of the project and should carry the strongest mathematical and computational narrative.

5.1 Statement of the encoding problem

- **Include.** Formulate the encoding problem precisely: given a module or presentation over a large or geometric parameter domain, construct a finite poset P, an encoded module over P, and a classifier or encoding map π from the ambient domain into the regions of P.
- **Include.** State the practical requirement that the encoded object retain enough information to support evaluation, comparison, invariant computation, and subsequent algebraic manipulations.

5.2 Region signatures and the finite-poset idea

- **Include.** Explain the region/signature philosophy: points in the ambient domain are grouped by their membership signatures with respect to the front-end generators, and these signatures determine the regions of the finite encoding.
- **Include.** Describe how the resulting finite poset structure is obtained from inclusion relations on signatures, producing uptight/signature-like posets in the sense suggested by the theory [Mil20].
- **Possible figures/tables.** A signature-region diagram in $\mathbb{Z}^2$ or $\mathbb{R}^2$.

5.3 Encoding finite fringe data

- **Include.** Explain the finite fringe case first because it is conceptually simplest: the input already lives over a finite poset, so the problem becomes one of standardization, caching, and producing a workflow-friendly result object.
- **Include.** Use this subsection to introduce the idea that all later encoding functions should ultimately return the same type of workflow payload: a finite poset, an encoded module, and a compiled classifier map.

5.4 Encoding $\mathbb{Z}^n$ flanges

- **Include.** This subsection should be detailed. Describe the discrete encoding path from flange presentations to finite region/signature posets, the classifier map on lattice points, and the construction of pushed-forward fringe or module data.
- **Include.** Explain why this is a mathematically natural computational realization of Miller's reduction from finitely determined $\mathbb{Z}^n$-modules to finite encodings and back [Mil20].
- **Include.** Discuss compiled encoding caches, encoding fingerprints, pushforward plans, and the idea that expensive geometry/combinatorics should be reusable across repeated workflow queries.
- **Possible figures/tables.** An end-to-end example: flange $\to$ region signatures $\to$ finite poset $P \to$ encoded module.

5.5 Encoding $\mathbb{R}^n$ PL and box presentations

- **Include.** Describe the two-backend story over $\mathbb{R}^n$: a box-specialized backend for axis-aligned finite-boundary data and a more general PL/polyhedral backend for broader geometric inputs.
- **Include.** Explain backend routing: when box structure is available, one can exploit coordinate grids and signature tables; when not, one falls back to a more general PL/polyhedral encoding path.
- **Include.** Explain point location, region representatives, boundary ambiguity handling, and the distinction between fast and verified modes where appropriate.
- **Comparison point.** This is a good place to emphasize that the library is not restricted to one ambient parameter class: the same post-encoding computations can start from either $\mathbb{Z}^n$ or $\mathbb{R}^n$ inputs once the finite encoding is built.

5.6 Data-to-encoding via the ingestion layer

- **Include.** Describe how raw data is turned into graded complexes or filtration-derived objects, then resolved into one of the representation formats above, and then encoded into a finite poset.
- **Include.** This subsection should make explicit that the new data-ingestion chapter is a major advance over the earlier thesis attempt: the library no longer presupposes that users arrive with already-packaged algebraic objects.
- **Include.** Name representative filtration families only briefly. The point is not to catalogue every filtration, but to show that the library now controls the entire route from data to encoded algebra.
- **Possible figures/tables.** A boxed workflow diagram from point cloud / graph / image to encoded module.

5.7 Common encodings for multiple modules

- **Include.** Explain the need for common encodings when two objects are to be compared, transported, or used as inputs to bifunctorial constructions.
- **Include.** Discuss the workflow-level support for encoding multiple inputs into a shared finite poset model and why that matters for Hom, Ext, comparisons, and benchmarks.

5.8 Compiled encodings, point location, and evaluation

- **Include.** Describe compiled encodings as a bridge between mathematical correctness and practical performance: users need a classifier map that can be applied repeatedly to ambient points without rebuilding geometry or searching naively.
- **Include.** Discuss point location for lattice and real inputs, region representative data, and why compiled evaluation is essential for invariants, featurizers, and visualization.

5.9 Geometry attached to an encoding map

- **Include.** Explain that the output of the encoding step is not only a finite poset and a classifier, but also a geometric decomposition of the ambient parameter space into constant regions. Once an encoding map is available, one can ask geometric questions about those

regions: their representatives, bounding boxes, volumes or weighted sizes inside a chosen window, adjacency relations, boundary size, and related shape descriptors.

- **Include.** Make clear that this geometric layer is backend-sensitive: over $\mathbb{Z}^n$ and box-based real encodings, much of the relevant geometry can be computed directly from coordinate thresholds and region signatures, while more general PL backends require a richer geometric interface.
- **Include.** State explicitly that this information is not a separate pre-encoding input type. It is produced by the encoding and then consumed later by invariant, featurization, and visualization layers.
- **Include.** Suggested mathematical emphasis: the encoding map turns ambient parameter space into a finite stratified object, and region geometry records quantitative information about the strata rather than about the original presentation generators.
- **Possible figures/tables.** A small example showing an encoding in $\mathbb{R}^2$ together with one or two labeled regions, their representatives, and a simple adjacency graph.

5.10 Correctness, diagnostics, and failure modes

- **Include.** List the kinds of guarantees the encoding layer should provide: well-defined region posets, validated dimensions, monomiality or support constraints where appropriate, and reproducible classifier behavior.
- **Include.** Discuss practical failure modes: region explosion, degenerate geometric boundaries, budget overflows in ingestion, and backend-incompatibility cases.
- **Include.** Explain why this explicit diagnostic surface is part of the library's value.

5.11 Complexity and why this chapter is the thesis core

- **Include.** Provide a narrative complexity discussion rather than a fully formal theorem-by-theorem analysis. Break cost into: front-end normalization, region construction, poset construction, pushforward/build of encoded modules, compiled point-location support, and cache reuse.
- **Include.** State clearly that this chapter is the main differentiator: once encoding is available, the library can support an unusually broad collection of exact algebraic computations that are difficult to expose in systems built only around barcode-level summaries.
- **Deliverable.** This section should end with a transition sentence of the form: from this point onward, the thesis works almost entirely inside the category of modules over a finite encoding poset.

6 Post-encoding algebra on finite posets

- **Target length.** 5–6 pages.
- **Purpose.** Show what becomes possible once modules live over finite encoding posets. The emphasis here is not on re-proving classical category theory, but on explaining the computational opportunities that finite encoding opens.

6.1 Encoded modules and morphisms as the internal working language

- **Include.** Introduce P-modules and morphisms over a finite poset P as the system's internal algebraic model.
- **Include.** Explain the role of cover-edge map stores, map-composition queries, cached evaluation along comparable pairs, and why finite posets make these operations practical.

6.2 Abelian-category constructions in code

- **Include.** Briefly explain that kernels, images, cokernels, coimages, pushouts, pullbacks, short exact sequences, equalizers, and related constructions are implemented concretely in the finite-poset module category.
- **Include.** Do not spend pages re-deriving abelian-category theory. Instead, explain why having these operations as first-class computational objects matters for persistence-style workflows.

6.3 Indicator resolutions, projective covers, and injective hulls

- **Include.** Describe the role of indicator modules and their resolutions: once a module is encoded, projective and injective constructions can be built from upsets and downsets in the finite poset.
- **Include.** Connect this explicitly back to Miller's equivalence of tameness viewpoints and the importance of indicator resolutions in the theory [Mil20].

6.4 Derived functors and complex-level structure

- **Include.** Introduce, at a very high level, the implemented derived-functor layer: Hom, Ext, Tor, RHom, derived tensor, hyper-Ext, hyper-Tor, spectral sequences, and related complex-level constructions.
- **Keep brief.** Do not define all of these in full. The target audience already knows what derived functors are. The thesis should instead explain *why a finite encoded module category makes these computations accessible.*
- **Include.** Suggested narrative: unlike software pipelines that discard structure after producing ranks or barcodes, TAMER-OP retains an honest algebraic object on a finite poset, so one can ask richer homological questions directly.

6.5 Change of posets, transport, and comparison

- **Include.** Explain that once encodings are explicit, one can pull back, push forward, compare modules across related posets, and build common refinement posets or product posets when necessary.
- **Include.** This should serve as a bridge between the algebra chapter and the later benchmark/comparison sections.

6.6 What computational opportunities this opens

- **Include.** This is the argumentative subsection. Explain, in prose, what kinds of new workflows become available because of the explicit encoding architecture: naturality tests, derived comparisons, exact resolution-based invariants, transport along encoding maps, module-level experiment objects, and more.

- **Comparison point.** Frame the contrast with barcode- or rank-only systems carefully: the issue is not merely that TAMER-OP has more functions in an API, but that its core representation after encoding is algebraically expressive enough to support them.

7 Invariants and summaries after encoding

- **Target length.** 3–4 pages.
- **Purpose.** Briefly introduce the invariant layer, organized by the principle that invariants factor through the finite encoding. The aim is breadth of orientation, not exhaustive theory.

7.1 Restricted Hilbert and rank-type invariants

- **Include.** Describe the restricted Hilbert function, the rank invariant, and the fact that these become straightforward to evaluate once point location and map evaluation on the encoding are available.
- **Include.** Use this subsection to emphasize that encoded modules support both direct queries at the finite-poset level and classifier-mediated queries back in the ambient parameter space.

7.2 Slice-based invariants and distances

- **Include.** Briefly introduce slice barcodes, matching-distance-style constructions, projected distances, and the fibered viewpoint.
- **Include.** The main point should be that the encoding supplies a common finite substrate on which these slice and comparison procedures can be built and cached.

7.3 Geometry of encoding regions and weighted summaries

- **Include.** Explain that many post-encoding summaries are not purely algebraic counts on the finite poset. They are algebraic quantities weighted by geometric information attached to the encoded regions. This is where region size, adjacency, boundary measure, aspect ratio, diameter, and related data enter the story.
- **Include.** Describe the basic role of region weights: they allow one to integrate finite-poset information back over the ambient parameter domain. Examples include weighted Hilbert summaries, support measures for loci where Betti or Bass data are nonzero, and aggregate statistics of regions carrying a prescribed algebraic value.
- **Include.** Clarify that "Euler-style" summaries mean constructions obtained from an Euler characteristic surface or from Mobius inversion of encoded rank or Hilbert data, producing signed point or rectangle measures. The thesis should define this carefully rather than using the term as shorthand.
- **Include.** Note that some geometric summaries are exact, while others may use sampled or approximate backends; this is the natural place to mention lightweight statistical uncertainty quantification, such as confidence intervals for sample-based support estimates.
- **Include.** This subsection should connect directly to the thesis-wide theme: encoding does not recreate a one-parameter barcode, but it does make it possible to combine exact algebraic information with computable region geometry in a controlled finite framework.
- **Possible figures/tables.** A compact table pairing algebraic quantities with the geometric data they use: support measure with region weights, interface summaries with adjacency,

shape summaries with aspect ratio/diameter, and signed measures with Euler or rank surfaces.

7.4 Mobius inversions, signed measures, and Euler-type summaries

- **Include.** Describe rectangle signed barcodes, point signed measures, and Euler signed measures as examples of invariants built from encoded rank or Euler surfaces.
- **Include.** Keep the discussion conceptual: these are examples of how encoded finite data can be transformed into compact, transportable summaries for analysis and learning.

7.5 Multiparameter persistence images and higher summaries

- **Include.** Briefly introduce the Carrière-style MPPI constructions and associated decomposition objects implemented in the invariant/featurizer layers.
- **Include.** Emphasize that these are not ad hoc extras: they are examples of a design where sophisticated summaries are layered on top of the encoded module object rather than replacing it [LS24].

7.6 How much theory to include here

- **Keep brief.** Do not turn this into a catalogue chapter. Give one paragraph per invariant family, one or two compact examples, and a short table summarizing input type, output type, and intended use. Save detailed formulas for the most relevant invariants only.
- **Possible figures/tables.** A compact summary table is likely better than many separate subsections.

8 Serialization, featurization, and visualization

- **Target length.** 4–5 pages.
- **Purpose.** Explain the cross-cutting infrastructure that makes the library usable as a research platform rather than only as a collection of algebra routines.

8.1 Serialization and reproducibility

- **Include.** Explain the importance of stable JSON-facing schemas for datasets, pipelines, encodings, flanges, PL fringes, and expensive derived artifacts.
- **Include.** Argue that serialization is mathematically important because encoded modules, resolution outputs, and feature artifacts should be inspectable, reproducible, and transportable between runs, machines, or downstream analysis stages.
- **Include.** A short schematic of what gets serialized at which pipeline stage would be useful.

8.2 Featurization as an experiment layer

- **Include.** Describe the featurizer-spec architecture: encoded modules, invariants, signed measures, projected distances, MPPI summaries, and resolution-based tables can all be transformed into feature vectors under one experiment runner.
- **Include.** Explain why this matters: a multiparameter persistence library should not force users to rebuild machine-learning and benchmarking infrastructure outside the algebra system.

- **Include.** Briefly mention batch execution, cache reuse, metadata sidecars, and experiment manifests as reproducibility features.

8.3 From geometric summaries to features and visualization

- **Include.** Add a short bridge subsection explaining that region-geometry data and geometry-aware invariant summaries naturally feed two downstream layers: feature construction and visualization. Weighted Hilbert summaries, support measures, signed measures, and region-shape statistics become inputs to featurizers, while the same encoded geometry also provides the substrate for visual exploration of regions, interfaces, and invariant surfaces.
- **Include.** This subsection should explicitly connect the previous chapter to the present one: the geometric information attached to an encoding is first used to build mathematically meaningful summaries, and then reused to make those summaries exportable, comparable, and visible.
- **Include.** Keep this short. Its job is to make the transition from post-encoding mathematics to experiment infrastructure and visualization feel conceptually natural.

8.4 Why these layers matter scientifically

- **Include.** Make the case that serialization and featurization are part of the mathematical-computational story, not merely software engineering garnish. They enable controlled experiments, benchmark reproducibility, downstream statistical analysis, and transparent artifact exchange.
- **Comparison point.** This is another place to explain how TAMER-OP differs from workflows that produce an invariant and then leave the rest of the analysis stack to the user.

8.5 Visualization engine

- **Include.** Treat the visualization engine as part of the intended thesis scope. Present it as a natural extension of the encoding-centered architecture: once one has encoded regions, modules, and invariant summaries, one can visualize them coherently from the same internal objects.
- **Include.** Suggested topics: region plots for $\mathbb{Z}^2$ and $\mathbb{R}^2$ encodings, finite-poset visualizations, slice and matching-distance diagnostics, signed-measure and MPPI visualizations, and benchmark/exploration dashboards.
- **Include.** If the full engine is not complete at drafting time, anchor the section by referencing the existing 2D visualization seed and then describe the generalized engine as the intended integrated layer.
- **Possible figures/tables.** This chapter should probably include the richest visual material in the thesis. Plan at least three figures here.

9 Performance engineering and benchmarks

- **Target length.** 5–6 pages.
- **Purpose.** Demonstrate that the library is computationally tractable on realistic inputs and that the encoding-centered architecture pays off in practice. This chapter should combine implementation-aware performance explanation with careful empirical evaluation.

9.1 Benchmark philosophy and fairness

- **Include.** State benchmark principles explicitly: fixed hardware/software environment, exact versioning, repeated runs, warmup handling, fair parameter choices, and clear separation between microbenchmarks and end-to-end benchmarks.
- **Include.** Say how comparisons to MULTIPERS will be framed: choose common tasks where both systems are genuinely solving comparable problems, then explain where the task scope diverges because TAMER-OP retains a richer encoded object.

9.2 Where time is spent internally

- **Include.** Explain the internal performance story: exact field linear algebra, sparse versus dense kernels, backend routing, Nemo versus Julia paths, modular shortcuts, autotuned thresholds, caching, and threading.
- **Include.** This subsection should interpret performance in architectural terms so that later benchmark plots have context.

9.3 Microbenchmarks

- **Include.** Plan a suite of small isolated benchmarks for the main internal stages. Suggested categories:
  (a) data ingestion and graded-complex construction,
  (b) encoding construction over $\mathbb{Z}^n$ and $\mathbb{R}^n$,
  (c) point location / compiled encoding evaluation,
  (d) map-composition and common-poset transport,
  (e) projective/injective resolution steps,
  (f) selected invariants,
  (g) serialization and featurization overhead.
- **Possible figures/tables.** A benchmark matrix or table with rows = tasks and columns = time / memory / notes.

9.4 End-to-end workflow benchmarks

- **Include.** Benchmark complete workflows from raw input to encoded module and onward to invariants or features. Suggested pipelines: point cloud $\to$ ingestion $\to$ encoding $\to$ rank/slice features; graph/image $\to$ ingestion $\to$ encoding $\to$ Euler/feature summaries; $\mathbb{Z}^n$ flange $\to$ encoding $\to$ Ext/Tor or resolution-based outputs.
- **Include.** This subsection is important because it shows that the system is not merely a collection of isolated kernels.

9.5 Comparison with MULTIPERS

- **Include.** Design a fair comparison around overlapping capabilities: data-to-invariant workflows, slice-based summaries, matching-distance-style computations, or other common tasks.
- **Include.** State explicitly where TAMER-OP is faster and where its architectural choice of explicit encoding changes the nature of the workflow being benchmarked.
- **Include.** Do not overstate. If some tasks are not apples-to-apples, say so and explain why.

- **Comparison point.** A useful framing sentence: TAMER-OP pays an up-front encoding cost in order to unlock a larger post-encoding algebraic toolkit; the benchmark chapter should measure both that cost and those downstream benefits.

9.6 Discussion of tractability

- **Include.** End the chapter by interpreting the results: what dataset sizes are currently comfortable, where region explosion is the main bottleneck, when the box-specialized backend helps, when exact arithmetic is affordable, and what the benchmarks imply for realistic use.
- **Deliverable.** This subsection should answer the practical question: *when should a user reasonably expect this encoding-centered architecture to be worth it?*

10 Conclusion and future work

- **Target length.** 1–2 pages.
- **Purpose.** Close the loop on the main thesis claim.
- **Include.** Summarize the central argument: explicit finite encoding is the step that makes a unified algebraic multiparameter pipeline possible.
- **Include.** Reiterate the major implementation consequences: unified front-end formats, exact finite-poset computations, richer homological algebra, reusable invariants, reproducible experiment layers, and a coherent visualization story.
- **Include.** List immediate future work succinctly: completion and expansion of the visualization engine, broader benchmark coverage, richer geometric backends, stronger interoperability, and potentially more advanced algebraic constructions.

A Section-to-codebase map

- **Purpose.** This appendix exists purely to make drafting easier. When writing a section later, consult the corresponding module families first.
- **Include. Pre-encoding representations:** `FiniteFringe.jl`, `FlangeZn.jl`, `PLPolyhedra.jl`, `PLBackend.jl`, `DataTypes.jl`.
- **Include. Data ingestion:** `DataFileIO.jl`, `DataIngestion.jl`, `Options.jl`.
- **Include. Encoding workflow:** `ZnEncoding.jl`, `EncodingCore.jl`, `Workflow.jl`, `Results.jl`.
- **Include. Geometry attached to encodings:** `RegionGeometry.jl`, with backend implementations spread across `PLBackend.jl`, `PLPolyhedra.jl`, and encoding-map-specific methods inside the invariant layer.
- **Include. Finite-poset module layer:** `Modules.jl`, `AbelianCategories.jl`, `ChangeOfPosets.jl`.
- **Include. Resolutions and derived functors:** `IndicatorResolutions.jl`, `DerivedFunctors.jl`, `ModuleComplexes.jl`, `ChainComplexes.jl`.
- **Include. Invariants:** `Invariants.jl`, together with `RegionGeometry.jl` for region-level geometric data and `Stats.jl` for lightweight uncertainty summaries used by sampled geometric estimates.
- **Include. Serialization and experiment infrastructure:** `Serialization.jl`, `Featurizers.jl`, `APISurface.jl`, `PosetModules.jl`.

- **Include. Visualization:** `Viz2D.jl` as the present seed, together with the forthcoming generalized visualization engine to be documented in the main text.
- **Include. Performance internals:** `FieldLinAlg.jl`, `public_api.jl`, `backend_routing.jl`, `qq_engine.jl`, `sparse_rref.jl`, `nonqq_engines.jl`, `autotune.jl`, `thresholds.jl`, and `linalg_thresholds.toml`.

B Planned figures and tables

- **Include. Figure 1:** Global architecture figure from raw data / presentations to encoded module to post-encoding layers.
- **Include. Figure 2:** A toy $\mathbb{Z}^2$ flange and its finite encoding.
- **Include. Figure 3:** A box/PL encoding example over $\mathbb{R}^2$.
- **Include. Figure 4:** Data-ingestion pipeline for one or two representative data types.
- **Include. Figure 5:** Post-encoding algebraic workflow (kernels/images/resolutions/Ext).
- **Include. Figure 6:** Invariant and region-geometry summary panel (rank surface, weighted support summary, signed measure, MPPI).
- **Include. Figure 7:** Visualization engine showcase panel.
- **Include. Table 1:** Front-end formats versus ambient domain versus encoded output.
- **Include. Table 2:** Implemented post-encoding algebraic operations.
- **Include. Table 3:** Implemented invariant families and intended use.
- **Include. Table 4:** Benchmark task matrix.

C Benchmark checklist

- **Include.** Record hardware, Julia version, thread count, BLAS thread count, and package versions.
- **Include.** Fix coefficient field and exactness mode for each benchmark.
- **Include.** Separate wall-clock time, memory, and one-time compilation effects.
- **Include.** Use the same datasets and same output target whenever comparing against MULTIPERS.
- **Include.** Retain raw benchmark logs and serialized artifacts for reproducibility.
- **Include.** Include at least one benchmark each for: ingestion, encoding, post-encoding algebra, invariant computation, featurization, and end-to-end workflow.

D Bibliography notes

- **Purpose.** The bibliography below is already a superset of the one in the earlier thesis draft. Before final submission, extend it further with any additional references used in the visualization chapter, MPPI discussion, and software comparison section.

- **Include.** In particular, add any final citations for the visualization engine once its scope is settled, and any additional benchmark-comparison references that become necessary during drafting.

References

[Mil20] E. Miller. *Homological algebra of modules over posets.* arXiv:2008.00063, 2020.

[Waa24] L. Waas. *Notes on abelianity of categories of finitely encodable persistence modules.* arXiv:2407.08666, 2024.

[Mil23] E. Miller. *Stratifications of real vector spaces from constructible sheaves with conical microsupport. Journal of Applied and Computational Topology*, 7(3):473–489, 2023. doi:10.1007/s41468-023-00112-1.

[KS18] M. Kashiwara and P. Schapira. *Persistent homology and microlocal sheaf theory. Journal of Applied and Computational Topology*, 2(1–2):83–113, 2018. arXiv:1705.00955.

[CB15] W. Crawley-Boevey. *Decomposition of pointwise finite-dimensional persistence modules. Journal of Algebra and Its Applications*, 14(5):1550066, 2015. doi:10.1142/S0219498815500668.

[ZC05] A. Zomorodian and G. Carlsson. *Computing persistent homology. Discrete & Computational Geometry*, 33:249–274, 2005. doi:10.1007/s00454-004-1146-y.

[CZ09] G. Carlsson and A. Zomorodian. *The theory of multidimensional persistence. Discrete & Computational Geometry*, 42:71–93, 2009. doi:10.1007/s00454-009-9176-0.

[CEH07] D. Cohen-Steiner, H. Edelsbrunner, and J. Harer. *Stability of persistence diagrams. Discrete & Computational Geometry*, 37(1):103–120, 2007. doi:10.1007/s00454-006-1276-5.

[ELZ02] H. Edelsbrunner, D. Letscher, and A. Zomorodian. *Topological persistence and simplification. Discrete & Computational Geometry*, 28:511–533, 2002. doi:10.1007/s00454-002-2885-2.

[Les15] M. Lesnick. *The theory of the interleaving distance on multidimensional persistence modules. Foundations of Computational Mathematics*, 15(3):613–650, 2015. doi:10.1007/s10208-015-9255-y.

[LW15] M. Lesnick and M. Wright. *Interactive visualization of 2-D persistence modules.* arXiv:1512.00180, 2015.

[RIVET] M. Lesnick and M. Wright. *RIVET: The rank invariant visualization and exploration tool* (software). `https://github.com/rivetTDA/rivet`.

[LS24] D. Loiseaux and H. Schreiber. *multipers: Multiparameter persistence for machine learning. Journal of Open Source Software*, 9(103):6773, 2024. doi:10.21105/joss.06773.

[CDSGO16] F. Chazal, V. de Silva, M. Glisse, and S. Y. Oudot. *The Structure and Stability of Persistence Modules.* SpringerBriefs in Mathematics. Springer, 2016. doi:10.1007/978-3-319-42545-0.

[Oud15] S. Y. Oudot. *Persistence Theory: From Quiver Representations to Data Analysis.* Mathematical Surveys and Monographs, Vol. 209. American Mathematical Society, 2015.

[EH10] H. Edelsbrunner and J. L. Harer. *Computational Topology: An Introduction.* American Mathematical Society, 2010.

[Mac98] S. Mac Lane. *Categories for the Working Mathematician*. Graduate Texts in Mathematics, Vol. 5. Springer, 2nd edition, 1998. doi:10.1007/978-1-4757-4721-8.

[Wei94] C. A. Weibel. *An Introduction to Homological Algebra*. Cambridge Studies in Advanced Mathematics, Vol. 38. Cambridge University Press, 1994.

[Eis95] D. Eisenbud. *Commutative Algebra with a View Toward Algebraic Geometry*. Graduate Texts in Mathematics, Vol. 150. Springer, 1995. doi:10.1007/978-1-4612-5350-1.

[MS05] E. Miller and B. Sturmfels. *Combinatorial Commutative Algebra*. Graduate Texts in Mathematics, Vol. 227. Springer, 2005. doi:10.1007/b138602.